





ππππ ππππππ

**\*\*Value Type\*\*** → stores the actual value.

**\*\*Reference Type\*\*** → stores a reference to an object.

**\*\*Stack\*\*** → usually used for local variables and method-related data.

**\*\*Heap\*\*** → used for objects created dynamically.

فهم الفرق بين الـ **\*\*Stack\*\*** والـ **\*\*Heap\*\*** سيساعدنا نفهم بشكل أفضل إزاي #C بتتعامل مع الـ memory وإيه اللي بيحصل لما نعمل copy للـ variables.

## Variables Allocation in Stack and Heap (C#)

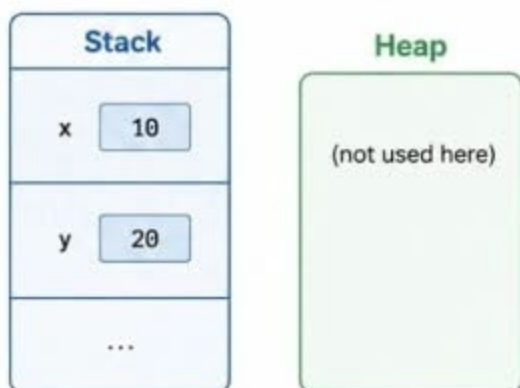
في (C#)، مكان تخزين الـ variable يعتمد على نوع البيانات: **Value Type** أو **Reference Type**

### 1) Value Types (e.g., int, double, bool, struct)

- الـ 10 = يُفَسِّها بتتخزن في الـ Stack.
- كل variable لـ نسخة مستقلة من الـ value.
- أمثلة: int, double, char, bool, struct

```
int x = 10;
int y = x; // y gets a copy of x
y = 20;
Console.WriteLine(x); // 10
Console.WriteLine(y); // 20
```

#### Memory Diagram (Value Types)



#### Key Point

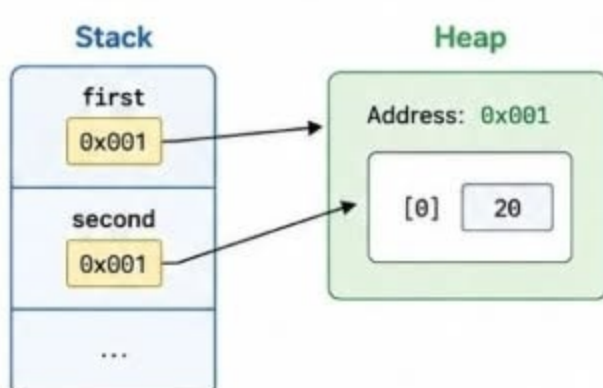
تغيير قيمة y لا يؤثر على x لأن كل واحد له مكانه الخاص في الـ Stack.

### 2) Reference Types (e.g., class, array, string)

- Reference في الـ Stack، بيخزن variable للـ object الموجود في الـ Heap.
- أكثر من variable ممكن يشير لنفس الـ object.
- أمثلة: class, array, string

```
int[] first = { 10 };
int[] second = first; // both point to
// the same object
second[0] = 20;
Console.WriteLine(first[0]); // 20
Console.WriteLine(second[0]); // 20
```

#### Memory Diagram (Reference Types)



#### Key Point

تغيير البيانات من خلال second يؤثر على first لأنهما يشيران لنفس الـ object في الـ Heap.

### In Short



#### Stack

سريع، حجمه محدود،  
variables للـ local يُستخدم  
والـ value types.



#### Heap

أكبر حجماً، أبطأ نسبياً،  
يُستخدم للـ objects.  
(reference types).



#### Value Type

يخزن القيمة نفسها في  
الـ Stack، وكل نسخة  
مستقلة.



#### Reference Type

يخزن Reference في الـ Stack  
والـ object نفسه في الـ Heap،  
أكثر من variable ممكن يشير  
لنفس الـ object.



Home



My Network



Post



Notifications



Jobs

 Search**Arwa Mahmoud** • You

Computer Science Student | October 6 Unive...

now • 

## Variables Allocation in Stack and Heap

لما بنتكلم عن **\*\*variables\*\*** في C، مهم نفهم ال **\*\*memory allocation\*\*** بيحصل إزاي، خصوصًا الفرق بين **\*\*Value Types\*\*** و **\*\*Reference Types\*\***.

### ### Value Types

ال **\*\*Value Type\*\*** بيخزن ال value نفسها. ومن أمثله `int`، `double`، و `bool`.

غالبًا ال `local variables` من ال **\*\*Value Types\*\*** بتكون موجودة في ال **\*\*Stack\*\***، وال `Stack` بيتميز إنه سريع في التعامل مع البيانات.

```
```csharp
int x = 10;
int y = x;
```
```

هنا `x` و `y` كل واحد فيهم عنده value مستقلة، فلو غيرنا `y`، قيمة `x` مش هتتغير.

### ### Reference Types

أما ال **\*\*Reference Type\*\*** زي `class` وال `array`، فال `object` نفسه بيكون في ال **\*\*Heap\*\***، وال `variable` بيحتوي على **\*\*reference\*\*** بيوصل لـ `object`.

```
```csharp
```

Post successful. [View post](#)

Home



My Network



Post



Notifications



Jobs



→↺excalidraw.com

🔒👤🖱️1📐2🔷3🔵4→5-6✎7A8🖼️9🔗0⋮

To move canvas, hold `Scroll wheel` or `Space` while dragging, or use the hand tool

Excalidraw+Share📄

1.Shortcut to comment and uncomment code in Visual Studio:  
  
To comment a selected block of code, press `Ctrl + K, Ctrl + C`.  
To uncomment it, press `Ctrl + K, Ctrl + U`.  
  
2.Runtime error vs Logical error:  
A runtime error happens while the program is running, and it may stop the program.  
  
Example: trying to divide by zero.  
A logical error means the program runs normally but gives the wrong result.  
  
Example: using `+` instead of `*` in a calculation  
  
3.Why naming conventions like PascalCase are important in C#:  
Naming conventions make the code easier to read and understand. They also make the code look organized and professional.  
  
For example: `StudentName` instead of `studentname`.  
  
4.Value types vs Reference types in memory:  
Value types store the actual value directly.  
Example: `int`, `double`, `bool`.  
Reference types store a reference to the object in memory. Example: `class`, `string`, and `arrays`.

5.Output of the code:  

```
int a = 2, b = 7;  
Console.WriteLine(a % b);
```

Output: 2  
Because `%` gives the remainder after division.  
2 / 7 has remainder 2, since 2 is smaller than 7.

6.Difference between `&&` and `&`:  
  
`&&` is the logical AND operator and is mainly used with conditions. It also uses short-circuit evaluation.  
  
`&` is the bitwise AND operator and works on the bits of numbers.  
  
Example: `5 & 3` compares their binary bits.  
  
7.Why explicit casting is required from `double` to `int`:  
Because `double` can contain decimal values, while `int` cannot. So C# requires us to explicitly say that we want to convert it.  
  
Example:  

```
double x = 5.8;  
int y = (int)x;
```

The value of `y` will be 5 because the decimal part is removed.

8.Exception when the input is invalid:  
  
If we use `int.Parse()` and the user enters something that is not a valid integer, a `FormatException` can occur.  
  
We can handle it using try-catch:  

```
try  
{  
    int x = int.Parse(Console.ReadLine());  
}  
catch (FormatException)  
{  
    Console.WriteLine("Invalid input");  
}
```

  
9.Value of `x` after execution:  

```
int x = 5;  
int y = ++x + x++;
```

The value of `x` = 7.  
First, `++x` increases `x` from 5 to 6 and uses 6.  
Then `x++` uses 6 first and increases `x` to 7 after that.  
  
So `y = 6 + 6 = 12`, and finally `x = 7`.

10.Compiled vs Interpreted languages, and C#  
A compiled language is translated into another form before running, while an interpreted language is executed more directly at runtime.  
C# is mainly a compiled language. The C# code is compiled into Intermediate Language (IL), then the CLR uses JIT (Just-In-Time) compilation to convert it into machine code when running.  
  
Compare implicit, explicit, Convert and Parse:  
Implicit casting: happens automatically when the conversion is safe.  
  
Example:  

```
int x = 10;  
double y = x;
```

Explicit casting: we write the type ourselves when there may be data loss.  
  
Example:  

```
double x = 10.5;  
int y = (int)x;
```

Convert: is used to convert between different data types and has methods like `Convert.ToInt32()`.  
  
Example:  

```
int x = Convert.ToInt32(10.5);
```

Parse: is mainly used to convert a string into a specific type.  
  
Example:  

```
int x = int.Parse("10");
```

  
In short:  
Implicit → automatic conversion  
Explicit → manual casting using (type)  
Convert → conversion using Convert methods  
Parse → converts a string into a type

32%+↶↷

26°C  
غائم جزئيًا

🔍 Search

ENG  
INTL

0:07  
ص ٢٠٢٦/٨/١٧